Practical Session: Statistical Inference and Regression

Estimation, Tests and Predictive Evaluation with Python

Youssef SALMAN

Academic Year 2026–2027

Goals of the Practical Session

This practical session connects statistical inference and regression to simulation, numerical calculations, and graphical diagnostics in Python.

At the end of the session, students should be able to examine sampling distributions; measure bias and MSE; compute likelihood estimates and confidence intervals; test hypotheses and study power; compare parametric and non-parametric methods; fit and evaluate regression models.

The required libraries are NumPy, SciPy, Matplotlib, and scikit-learn.

1 Required Libraries

We use NumPy for arrays and simulation, SciPy for inference, Matplotlib for visualization, and scikit-learn for regression metrics. A fixed generator seed makes the simulation reproducible.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

rng = np.random.default_rng(123)
```

2 Sampling Distributions

For i.i.d. observations with mean μ and standard deviation σ, the sample mean has $E(\bar{X}) = \mu$ and $\mathrm{SE}(\bar{X}) = \sigma/\sqrt{n}$. Compare repeated samples of size $n = 5$ and $n = 50$ from an exponential population with mean 10.

```
mu, n_reps = 10.0, 5000
fig, ax = plt.subplots()
```

```
for n in [5, 50]:
    samples = rng.exponential(scale=mu, size=(n_reps, n))
    means = samples.mean(axis=1)
    print(f"n={n}, mean={means.mean():.3f}")
    print(f"empirical SE={means.std(ddof=1):.3f}, theoretical SE={mu/np.sqrt(n)
        :.3f}")
    ax.hist(means, bins=45, density=True, alpha=.5, label=f"n={n}")
ax.axvline(mu, color="black", linestyle="--", label="Population mean")
ax.set(xlabel="Sample mean", ylabel="Density", title="Sampling distributions")
ax.legend(); plt.show()
```

3 Bias and Mean Squared Error

For each sample compute the empirical variance with divisor n and the corrected variance with divisor $n-1$. The population variance of this normal model is $\sigma^2 = 16$. Compare bias and MSE across repeated samples.

```
n, reps, sigma = 8, 10000, 4.0
samples = rng.normal(loc=20, scale=sigma, size=(reps, n))
for ddof in [0, 1]:
    estimates = samples.var(axis=1, ddof=ddof)
    bias = np.mean(estimates - sigma**2)
    mse = np.mean((estimates - sigma**2)**2)
    print(f"ddof={ddof}, mean={estimates.mean():.3f}")
    print(f"bias={bias:.3f}, MSE={mse:.3f}")
plt.hist(samples.var(axis=1, ddof=0), bins=55, alpha=.5, label="divisor n")
plt.hist(samples.var(axis=1, ddof=1), bins=55, alpha=.5, label="divisor n-1")
plt.axvline(sigma**2, color="black", linestyle="--", label="True variance")
plt.xlabel("Variance estimate"); plt.ylabel("Frequency")
plt.legend(); plt.show()
```

4 Maximum Likelihood Estimation

For a Bernoulli sample with k successes in n observations, $L(p) = p^k(1-p)^{n-k}$ and $\hat{p}_{\text{MLE}} = k/n$. Compare the numerical likelihood maximum to the analytic estimate.

```
observations = rng.binomial(1, .35, size=80)
k, n = observations.sum(), len(observations)
p_hat = k / n
p_grid = np.linspace(.001, .999, 500)
log_likelihood = stats.binom.logpmf(k, n, p_grid) # differs by a constant
print("Successes:", k, "MLE:", p_hat)
print("Grid maximum:", p_grid[np.argmax(log_likelihood)])
plt.plot(p_grid, log_likelihood)
plt.axvline(p_hat, color="red", linestyle="--", label="MLE")
plt.xlabel("p"); plt.ylabel("Log likelihood (up to a constant)")
plt.legend(); plt.show()
```

5 Confidence Interval for a Mean

When the population standard deviation is unknown and observations are normal, use $\bar{x} \pm t_{n-1,1-\alpha/2} s/\sqrt{n}$. Construct a 95% interval for battery lifetime.

```
battery = rng.normal(loc=1000, scale=120, size=25)
n = len(battery)
mean, s = battery.mean(), battery.std(ddof=1)
interval = stats.t.interval(.95, df=n-1, loc=mean, scale=s/np.sqrt(n))
print(f"n={n}, sample mean={mean:.2f}, sample SD={s:.2f}")
print("95% t confidence interval:", interval)
```

6 Coverage of Confidence Intervals

A 95% confidence procedure covers the fixed true mean in about 95% of repeated samples, under its assumptions. Simulate independent normal samples and plot the first 35 intervals.

```
true_mu, true_sd, n, reps = 100.0, 15.0, 20, 1500
samples = rng.normal(true_mu, true_sd, size=(reps, n))
means = samples.mean(axis=1)
ses = samples.std(axis=1, ddof=1) / np.sqrt(n)
critical = stats.t.ppf(.975, df=n-1)
lo, hi = means - critical*ses, means + critical*ses
covered = (lo <= true_mu) & (true_mu <= hi)
print("Empirical 95% coverage:", covered.mean())
fig, ax = plt.subplots(figsize=(8, 6))
for i in range(35):
    ax.plot([lo[i], hi[i]], [i, i], color="C0" if covered[i] else "C3")
ax.axvline(true_mu, color="black", linestyle="--")
ax.set(xlabel="Population mean", ylabel="Simulation", title="First 35
    confidence intervals")
plt.show()
```

7 Confidence Interval for a Proportion

For a survey of $n = 200$ people with $k = 120$ positive responses, compute the Wald 95% interval and the Wilson interval. The Wilson method is often more reliable near 0 or 1.

```
from scipy.stats import norm
k, n = 120, 200
p_hat = k / n
z = norm.ppf(.975)
margin = z * np.sqrt(p_hat*(1-p_hat)/n)
print("Wald 95% interval:", (p_hat-margin, p_hat+margin))
center = (p_hat + z*z/(2*n)) / (1+z*z/n)
half = z*np.sqrt(p_hat*(1-p_hat)/n + z*z/(4*n*n)) / (1+z*z/n)
```

```
print("Wilson 95% interval:", (center-half, center+half))
```

8 One-Sample Hypothesis Test

Test $H_0 : \mu = 100$ against $H_1 : \mu > 100$ using a one-sided Student test. A p-value is computed under H_0; it is not the probability that H_0 is true.

```
x = rng.normal(loc=105, scale=12, size=36)
result = stats.ttest_1samp(x, popmean=100, alternative="greater")
print("Sample mean:", x.mean())
print("t statistic:", result.statistic, "p-value:", result.pvalue)
print("Decision at alpha=0.05:", "Reject H0" if result.pvalue <= .05 else "Do
    not reject H0")
```

9 Significance and Power

At a fixed 5% significance level, simulate tests when the null mean is true ($\mu = 100$) and when a small change is present ($\mu = 103$). Rejection under the null estimates Type I error; rejection under the alternative estimates power.

```
n, reps, sigma = 30, 3000, 12.0
for actual_mu in [100.0, 103.0, 107.0]:
    samples = rng.normal(actual_mu, sigma, size=(reps, n))
    tests = stats.ttest_1samp(samples, popmean=100, axis=1, alternative="
        greater")
    rate = (tests.pvalue <= .05).mean()
    print(f"Actual mu={actual_mu}, rejection rate={rate:.3f}")
```

10 Parametric and Non-Parametric Tests

Compare two independent groups using Welch's t-test and the two-sided Mann–Whitney rank test. These tests target different aspects of the group distributions; discuss whether differences in shape affect interpretation.

```
a = rng.normal(loc=20, scale=4, size=35)
b = rng.normal(loc=23, scale=5, size=35)
welch = stats.ttest_ind(a, b, equal_var=False)
rank_test = stats.mannwhitneyu(a, b, alternative="two-sided")
print("Group means:", a.mean(), b.mean())
print("Welch t-test p-value:", welch.pvalue)
print("Mann-Whitney p-value:", rank_test.pvalue)
plt.boxplot([a, b], tick_labels=["Algorithm A", "Algorithm B"])
plt.ylabel("Response time"); plt.show()
```

11 Simple Linear Regression

Model a response as $Y_i =$
$beta_0 +$
$beta_1 x_i +$
$varepsilon_i$. Fit the ordinary least squares line, inspect residuals, and compare the fitted slope with the simulation value 2.5.

```
x = np.linspace(0, 10, 80)
y = 4 + 2.5*x + rng.normal(0, 3, size=len(x))
X = x.reshape(-1, 1)
model = LinearRegression().fit(X, y)
y_hat = model.predict(X)
print("Intercept:", model.intercept_, "Slope:", model.coef_[0])
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].scatter(x, y, s=15); axes[0].plot(x, y_hat, color="red")
axes[0].set(xlabel="x", ylabel="y", title="OLS fit")
axes[1].scatter(y_hat, y-y_hat, s=15); axes[1].axhline(0, color="red")
axes[1].set(xlabel="Fitted value", ylabel="Residual", title="Residuals")
plt.tight_layout(); plt.show()
```

12 R-Squared and Prediction Errors

Compute R^2, MSE, RMSE, and MAE. For predictive assessment, fit on a training set and measure error on held-out observations, using the same target units.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.25,
    random_state=42)
model = LinearRegression().fit(X_train, y_train)
pred = model.predict(X_test)
print("Test R^2:", r2_score(y_test, pred))
print("Test MSE:", mean_squared_error(y_test, pred))
print("Test RMSE:", np.sqrt(mean_squared_error(y_test, pred)))
print("Test MAE:", mean_absolute_error(y_test, pred))
```

13 Multiple Linear Regression

Predict house price using area, age and distance from the center. Interpret the area coefficient while holding age and distance fixed. Coefficients are synthetic, so the values are for method practice only.

```
n = 180
area = rng.uniform(50, 180, n)
age = rng.uniform(0, 35, n)
distance = rng.uniform(1, 18, n)
price = 30 + 2.2*area - .8*age - 1.5*distance + rng.normal(0, 18, n)
```

```
X_house = np.column_stack([area, age, distance])
X_train, X_test, y_train, y_test = train_test_split(X_house, price, test_size
    =.25, random_state=42)
reg = LinearRegression().fit(X_train, y_train)
pred = reg.predict(X_test)
print("Intercept:", reg.intercept_)
print("Coefficients (area, age, distance):", reg.coef_)
print("Test RMSE:", np.sqrt(mean_squared_error(y_test, pred)))
print("Test R^2:", r2_score(y_test, pred))
plt.scatter(y_test, pred, alpha=.7)
lims = [min(y_test.min(), pred.min()), max(y_test.max(), pred.max())]
plt.plot(lims, lims, "r--")
plt.xlabel("Observed price"); plt.ylabel("Predicted price")
plt.title("Multiple regression: held-out predictions")
plt.show()
```

14 Tasks for Students

1. Change the sample sizes in the sampling-distribution simulation. How does the empirical standard error compare with $\sigma/\sqrt{n}$?

2. For the variance estimators, compare bias and MSE at $n = 5, 20, 100$. Which estimator has smaller MSE in each case?

3. Repeat the confidence-interval coverage experiment using exponential data with $n = 5$ and $n = 50$. Explain the change in coverage.

4. For the hypothesis test, compare one-sided and two-sided alternatives. Explain why the p-values differ.

5. Vary the effect size and sample size in the power simulation. Plot power against sample size.

6. Introduce an outlier into the simple regression data. Compare MSE and MAE, and inspect its effect on the fitted line.

7. Add a fourth irrelevant predictor to multiple regression. Compare training and test performance.